Adam writing benchmark: ten trials per condition

Results

Paragraph	Condition	Grammarly / 100	Words
Method	RAW	93.2 ± 2.8	145.9 ± 4.4
Method	PIRA	**95.7 ± 2.1**	143.2 ± 4.2
Experiment	RAW	90.6 ± 3.4	191.1 ± 12.2
Experiment	PIRA	91.0 ± 3.7	**116.6 ± 7.5**

Values are mean ± sample standard deviation across ten independent conversations per condition. Both conditions use GPT-6.1-Sol/high and matched task prompts. The PIRA condition uses the previous writing rule. Word counts use whitespace-separated tokens in the captured Markdown, including math.

Writing assessment

Method: both conditions follow a stable gradient-to-update explanation. PIRA more often separates definitions into readable sentences; correction-and-update sentences remain dense in both.

Experiment: PIRA reaches the comparisons sooner and repeats less. Raw often delays the results with protocol details and uses awkward phrases such as “nearby lower training costs.” PIRA still has occasional heavy qualifiers and long comparison sentences.

The examples below are selected solely by Grammarly score within each condition. Ties use the earliest trial.

Method examples

RAW — highest score

Trial 01 · Grammarly 96/100 · 146 words

Adam updates parameters using bias-corrected exponential moving averages of the stochastic gradient and its elementwise square (Kingma and Ba, 2015). At step $t \geq 1$, let g_t denote the stochastic gradient evaluated at the current parameters θ_{t-1}. Starting from $m_0 = v_0 = 0$, the method computes $m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t$ and $v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2$, where $\beta_1, \beta_2 \in [0, 1)$ control the respective decay rates; m_t estimates the gradient's first moment (mean), while v_t estimates its second raw moment (mean squared gradient). Zero initialization biases these averages toward zero during early steps, particularly when the decay rates are close to one, so Adam forms the corrected estimates $\widehat{m}_t = m_t/(1-\beta_1^t)$ and $\widehat{v}_t = v_t/(1-\beta_2^t)$. The parameter update is then $\theta_t = \theta_{t-1} - \alpha\widehat{m}_t/(\sqrt{\widehat{v}_t}+\epsilon)$, where α is the stepsize and $\epsilon > 0$ is a small denominator stabilizer. All vector operations, including squaring, square roots, and division, are elementwise: the corrected first moment supplies the smoothed gradient, and the corrected second moment supplies coordinatewise scaling.

RAW — lowest score

Trial 09 · Grammarly 88/100 · 149 words

Adam constructs parameter updates from exponential moving averages of the stochastic gradient and its elementwise square (Kingma and Ba, 2015). At iteration $t \geq 1$, let g_t be the stochastic gradient evaluated at the current parameters θ_{t-1}. Starting from $m_0 = v_0 = 0$, the averages are updated as $m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t$ and $v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2$, where $\beta_1, \beta_2 \in [0, 1)$ control the respective decay rates. Here, m_t estimates the gradient's first moment (mean), while v_t estimates its second raw moment (mean squared gradient). Zero initialization biases these averages toward zero during early iterations, especially when the decay rates are close to one. Adam compensates for this initialization bias using $\widehat{m}_t = m_t/(1-\beta_1^t)$ and $\widehat{v}_t = v_t/(1-\beta_2^t)$, then updates the parameters according to $\theta_t = \theta_{t-1} - \alpha\widehat{m}_t/(\sqrt{\widehat{v}_t}+\epsilon)$, where α is the stepsize and $\epsilon > 0$ is a small denominator stabilizer. Thus, the bias-corrected first moment supplies the smoothed gradient, and the bias-corrected second moment determines its coordinatewise scaling; squaring and all vector operations are elementwise.

PIRA — highest score

Trial 05 · Grammarly 100/100 · 145 words

Adam (Kingma and Ba, 2015) maintains estimates of the first and second raw moments of the stochastic gradient. At step t, let g_t denote the stochastic gradient evaluated at the current parameters θ_{t-1}. Starting from $m_0 = v_0 = 0$, Adam computes the exponential moving averages $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$ and $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$, where $\beta_1, \beta_2 \in [0, 1)$ control the respective decay rates. These averages estimate the gradient's mean and mean square, respectively. Zero initialization biases both estimates toward zero during early steps, especially when the decay rates are close to one. Adam corrects this initialization bias using $\hat{m}_t = m_t/(1-\beta_1^t)$ and $\hat{v}_t = v_t/(1-\beta_2^t)$, then updates the parameters as $\theta_t = \theta_{t-1} - \alpha \hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon)$, where α is the stepsize and $\epsilon > 0$ is a small denominator stabilizer. The corrected first moment supplies the smoothed gradient, while the square root of the corrected second raw moment sets its coordinatewise scale. Squaring, square roots, division, and all other vector operations are elementwise.

PIRA — lowest score

Trial 02 · Grammarly 92/100 · 138 words

At step t, Adam (Kingma and Ba, 2015) evaluates the stochastic gradient g_t at the current parameters θ_{t-1}. Starting from $m_0 = v_0 = 0$, it maintains exponential moving averages $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$ and $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$, which estimate the gradient's first moment (mean) and second raw moment (mean squared gradient), respectively. The decay rates $\beta_1, \beta_2 \in [0, 1)$ control the weighting of past estimates and the current gradient. Zero initialization biases both averages toward zero during early steps, especially when the decay rates are close to one. Adam compensates for this initialization bias using $\hat{m}_t = m_t/(1-\beta_1^t)$ and $\hat{v}_t = v_t/(1-\beta_2^t)$, then updates the parameters as $\theta_t = \theta_{t-1} - \alpha \hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon)$, where α is the stepsize and $\epsilon > 0$ is a small denominator stabilizer. All vector operations, including squaring, square roots and division, are elementwise. Thus, the bias-corrected first moment supplies the smoothed gradient, while the bias-corrected second moment determines its coordinatewise scaling.

Experiment examples

RAW — highest score

Trial 03 · Grammarly 94/100 · 193 words

Figure 1 compares optimizer convergence for logistic-regression classification on MNIST and IMDB, plotting training negative log likelihood against passes through the full dataset (Kingma and Ba, 2015). The comparisons use identical parameter initializations, the best hyperparameters from dense-grid searches, and the stepsize schedule $\alpha_t = \alpha/\sqrt{t}$. On MNIST (left), L2-regularized multiclass logistic regression operates directly on 784-dimensional image vectors, yielding a convex objective, and is trained with minibatches of 128. Adam and SGD with Nesterov momentum exhibit similar convergence: their training costs decrease sharply at first and then improve gradually, whereas Adagrad decreases more slowly and retains a higher cost over the plotted run. On IMDB (right), movie reviews are represented by highly sparse bag-of-words vectors over the 10,000 most frequent words, with 50% dropout applied to the features to prevent overfitting. This panel compares Adam, Adagrad, RMSProp, and Nesterov SGD, all with dropout. Adam converges as fast as Adagrad, with both reaching nearby lower training costs while Nesterov SGD remains substantially higher; Adam thus exploits the sparse features to converge faster than SGD with momentum. Across the two tasks, Adam achieves tied-best convergence, matching Nesterov SGD on MNIST and Adagrad on IMDB.

RAW — lowest score

Trial 06 · Grammarly 84/100 · 177 words

Figure 1 evaluates logistic-regression optimization on MNIST and IMDB by plotting training negative log likelihood against passes through the full dataset (Kingma and Ba, 2015). Comparisons share parameter initialization, use the best hyperparameters from dense-grid searches, and adopt the stepsize schedule $\alpha_t = \alpha/\sqrt{t}$. On MNIST (left), L2-regularized multiclass logistic regression is trained on 784-dimensional image vectors with minibatches of 128, yielding a convex objective. Adam and SGD with Nesterov momentum show similar convergence: their training costs drop sharply initially and then decrease more gradually, whereas Adagrad improves more slowly and retains a higher cost over the plotted run. On IMDB (right), reviews are represented by sparse bag-of-words vectors over the 10,000 most frequent words, with 50% dropout on the input features to prevent overfitting; Adam, Adagrad, RMSProp and Nesterov SGD are all compared with dropout. Here Adam exploits the sparse features to converge as fast as Adagrad, with both reaching similarly low training costs while Nesterov SGD remains substantially higher. Adam therefore matches the best convergence on both tasks, alongside Nesterov SGD on MNIST and Adagrad on IMDB.

PIRA — highest score

Trial 03 · Grammarly 94/100 · 110 words

Figure 1 compares logistic-regression convergence on MNIST and IMDB through training negative log likelihood plotted against passes through the data (Kingma and Ba, 2015). Under matched initialization, with each optimizer evaluated at its best grid-searched hyperparameters and the stepsize schedule $\alpha_t = \alpha/\sqrt{t}$, Adam matches the best convergence on both datasets. On MNIST (left), L2-regularized multiclass logistic regression uses image vectors directly. Adam converges similarly to SGD with Nesterov momentum, and both reduce training cost faster than Adagrad. On IMDB (right), classification uses highly sparse bag-of-words inputs with 50% feature dropout. Adam converges as fast as Adagrad, with both reaching similar low training costs while Nesterov SGD remains substantially higher.

PIRA — lowest score

Trial 06 · Grammarly 84/100 · 112 words

In the logistic-regression experiments of Figure 1, Adam matches the fastest convergence on both MNIST and IMDB, as measured by training negative log likelihood over passes through the data (Kingma and Ba, 2015). The optimizers share the same parameter initialization and use their best hyperparameter settings from a dense grid search. On MNIST (left), Adam and SGD with Nesterov momentum reduce the cost at similar rates for L2-regularized multiclass classification of image vectors, both converging faster than Adagrad. On IMDB (right), where sparse bag-of-words features are subjected to 50% dropout, Adam converges as fast as Adagrad and reaches a similarly low training cost, while Nesterov SGD retains a substantially higher training cost.